B.Sc. (Hons) in Software Development

Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

TRiM - A Multi Tenant SaaS Application

By
Ariel C. Nunes

April 6, 2026

Minor Dissertation

Department of Computer Science & Applied Physics,
School of Science & Computing,
Atlantic Technological University (ATU), Galway.

Contents

1 Introduction **2**
1.1 Project Repository 2
1.2 Context 2
1.3 Project Objectives 3
1.4 Dissertation Focus 4
1.5 Success Metrics 4
1.6 Chapters Overview 5

2 Methodology **7**
2.1 Development Approach 7
2.2 Generative AI Usage 9
2.3 Research Approach 10
2.4 Evaluation Methodology 12

3 Technology Review **14**
3.1 Multi-Tenant Database Architectures 14
3.2 Related Works 17

4 System Design **19**
4.1 System Overview 19
4.2 Multi-Tenant Database Design 22
4.3 Multi-Tenant Implementation 25
4.4 Business Operations 27
4.5 Authentication & Access Control 32
4.6 Frontend Architecture 35
4.7 Deployment Architecture 35

5 System Evaluation **36**
5.1 Performance Evaluation 36
5.1.1 Database Gaitling Benchmarking - Load Testing 36
5.2 Security Evaluation 36

5.2.1 Tenant Isolation Verification 36
5.3 Critical Analysis 36
5.3.1 Strengths 36
5.3.2 Limitations 36

6 Conclusion **37**
6.1 Future Work 37

List of Figures

2.1 GitHub Projects Kanban board used to track development progress. 7
2.2 Example user story with acceptance criteria, priority, and technical notes. 9
2.3 Benchmarking API in IntelliJ IDE. 12
2.4 Example Gatling report showing per-request execution counts and response time percentiles. 13

3.1 Multi-tenant database isolation patterns 15

4.1 High-level system architecture diagram. 20
4.2 Entity-Relationship diagram of the PostgreSQL database schema. . 23

List of Tables

4.1 Summary of API endpoint groups. 22
4.2 Three independent layers of tenant data isolation. 25
4.3 Role requirements for protected API endpoint groups. 34

Chapter 1

Introduction

1.1 Project Repository

The source code for this project is available at `https://github.com/ArielCNunes/TRiM-FYP`. The repository is organised into three top-level directories:

- **`backend/`**: Contains the Spring Boot REST API, structured as a standard Maven project. Key packages include `controller`, `service`, `repository`, `entity`, and `tenant`, reflecting the layered architecture and multi-tenant logic at the core of the system. Supporting packages handle payment processing, email notifications, booking reminders, and test data seeding.
- **`frontend/`**: A React application built with TypeScript and Vite, styled with Tailwind CSS. The source is organised into `pages`, `components`, `features`, `hooks`, and `store` directories, separating UI concerns from state management and API integration.
- **`docs/`**: Contains this dissertation in LaTeX format, along with records of client and supervisor meetings maintained throughout the project.

1.2 Context

Small businesses such as barber shops that rely solely on walk-in customers face several operational challenges: limited business insights, customer friction when services are unavailable at their preferred time, and the burden of manual bookkeeping. Digitalising these operations bridges the gap between the business and its customers, leading to a more established market presence and streamlined day-to-day management.

TRiM is a platform designed to support the full range of barber shop operations, from customer bookings and payment processing to employee management and business analytics. Rather than serving a single business, *TRiM* is built to allow multiple businesses to operate on the platform independently, each with its own isolated data and user base. One instance of the software serves multiple businesses, this is known as multi-tenant architecture, and it is the core focus of this dissertation.

1.3 Project Objectives

The objective of this project is to deliver a real and robust software solution to local service-based businesses. The following goals have been defined:

- **Customer Booking System**: Enable customers to book services online, select their preferred barber, choose from categorized services, and pick available time slots while the system prevents double bookings and scheduling conflicts.
- **Payment Integration**: Integrate Stripe for secure online payment processing, supporting both full online payments and deposit-based bookings where customers pay a percentage upfront and settle the outstanding balance in-shop, driven by real client requirements.
- **Business Administration**: Provide business owners with a comprehensive admin dashboard to manage service categories, individual services with pricing and duration, and employee (barber) profiles. Include analytics such as total revenue, booking statistics, and popular services.
- **Barber Management**: Allow barbers to manage their own availability by setting weekly working hours, defining break periods, and viewing/managing their upcoming bookings with the ability to confirm, complete, or handle cancellations and no-shows.
- **Customer Management**: Enable administrators to view customer history, track no-show counts, and blacklist problematic customers to protect business operations.
- **Multi-Tenant Architecture**: Design the platform to support multiple independent businesses, each with their own subdomain (business slug), isolated data, and separate admin/barber/customer user bases.

- **Role-Based Access Control**: Implement distinct user roles (CUSTOMER, BARBER, ADMIN) with appropriate permissions and tailored interfaces for each role's specific needs.

1.4 Dissertation Focus

This dissertation focuses primarily on the architectural decisions behind *TRiM*, particularly the database design patterns and data isolation techniques employed. Through *TRiM*, we explore how to build a multi-tenant software application that is both scalable and robust, leveraging efficient design patterns that allow the system to grow without significant cost or performance overhead. We also investigate how multiple businesses can share the same platform whilst maintaining strong performance guarantees and strict security boundaries.

1.5 Success Metrics

From a functional standpoint, the system must handle all core operations reliably: bookings should complete without conflicts, payments should process securely, and the admin dashboard should provide accurate insights into business operations.

Beyond functionality, performance represents a critical area of evaluation. The system will be benchmarked under various conditions to assess its resilience. This includes measuring response times for key API endpoints, evaluating database performance as the number of tenants and bookings increases, and verifying that heavy usage by one business does not degrade the experience for others. Tenant isolation extends beyond security considerations, it also ensures fair distribution of system resources across all users.

Security constitutes another essential metric. With multiple businesses and their customer data at stake, the system must demonstrate robust protection mechanisms. Cross-tenant data leakage would represent a critical failure, necessitating thorough testing of all isolation mechanisms to prevent such occurrences.

Finally, usability remains a key consideration. The primary objective of *TRiM* is to streamline operations for business owners and their customers. An unintuitive interface or cumbersome booking flow would undermine the value of the underlying architecture.

1.6 Chapters Overview

Chapter 2: Methodology

Outlines the development approach used. It describes the agile workflow with a Kanban board in GitHub Projects, the user stories used to gather requirements, and the feedback loop with weekly supervisor meetings and regular client demonstrations with V7 Barbers. The chapter also covers the tools used for development, the role of generative AI, testing, and the evaluation strategy built around Gatling load testing and seeded test data.

Chapter 3: Technology Review

Looks at the three primary multi-tenant database architectures: separate databases, separate schemas, and shared schemas. Analyses their trade-offs in terms of isolation, cost, and scalability. The chapter surveys data isolation techniques including filter-based and permission-based approaches such as PostgreSQL Row-Level Security, and reviews related work on adaptive schema design, and recent benchmarking frameworks. It identifies the gap that motivates this project: a lack of empirical evidence on combining shared-schema designs with database-level security enforcement.

Chapter 4: System Design

Presents the architecture of *TRiM*, from the RESTful API and entity-relationship model to the multi-tenant implementation built on subdomain-based routing, tenant-aware schema design, and Row-Level Security. It details core operational flows including booking and payment processing with Stripe, race condition prevention, JWT-based authentication with role-based access control, and cross-subdomain token exchange. The chapter also covers the React frontend architecture and the Docker-based deployment strategy.

Chapter 5: System Evaluation

Evaluates the system against the success metrics defined in this chapter. Performance is assessed through Gatling load tests that measure API response times and database behaviour as tenant and booking volumes increase. Security is verified through tenant isolation testing to confirm that no cross-tenant data leakage occurs. The chapter concludes with a critical analysis of the system's strengths and the limitations of the current design.

Chapter 6: Conclusion

Reflects on the extent to which the project objectives were met, sums up the key findings from the system evaluation, and discusses the broader implications of the architectural choices made. It concludes with directions for future work, including potential improvements to the platform and areas where the multi-tenant design could be extended.

Chapter 2

Methodology

2.1 Development Approach

Agile Workflow

The development of *TRiM* followed an agile methodology, incorporating several techniques to ensure continuous progress and stakeholder alignment. A Kanban board was maintained using GitHub Projects to track all development work. Each item represented a unit of work from a user story and moved through *Backlog*, *In Progress*, and *Done* columns as development progressed, see Figure 2.1.

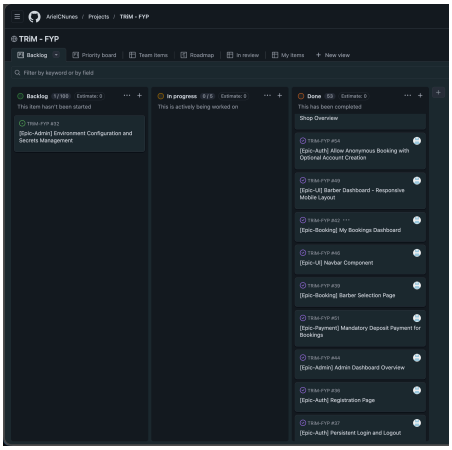

Figure 2.1: GitHub Projects Kanban board used to track development progress.

Weekly meetings were held with the project supervisor to review progress, discuss technical decisions, and receive guidance. In parallel, regular meetings were carried out with the client, Jean, the owner of V7 Barbers. All meetings were documented for reference.

Version Control and Development Tools

All source code was managed using Git and hosted on GitHub. Commits were made frequently to maintain a clear history of changes throughout the project. Development was carried out primarily using IntelliJ IDEA for the Spring Boot backend and Visual Studio Code for the React frontend. GitHub Codespaces was also used on occasion for remote development when working from a different machine. Docker Compose was used locally to run the PostgreSQL database alongside the application during development.

Requirements Gathering

Working with a real client was key to this project. Jean, the owner of V7 Barbers, participated in regular meetings that served as the primary mechanism for gathering and validating requirements. The first few sessions focused on understanding the day-to-day challenges of running a walk-in-only barber shop: limited visibility into peak times, no way to manage no-show customers, and the burden of manual bookkeeping. These conversations shaped the initial user stories and ensured the system addressed genuine business problems.

As development progressed, client meetings shifted more towardrs demonstration. Live walkthroughs of the system prompted feedback that hadn't been thought of before like deposit-based payments, where customers pay a percentage upfront and settle the balance in-shop, and customer blacklisting to protect the business from repeated no-shows.

Requirements were captured as user stories, each documented as a GitHub Issue following the same structure. Every user story included four sections: a **User Story** statement written in the standard "As a [role], I want [feature], so that [benefit]" format; a set of **Acceptance Criteria** expressed using Given/When/Then conditions; a **Priority** classification using the MoSCoW method (Must Have, Should Have, Could Have); and **Technical Notes** outlining relevant implementation details such as API endpoints and UI behaviour. Figure 2.2 illustrates a user story from the project.

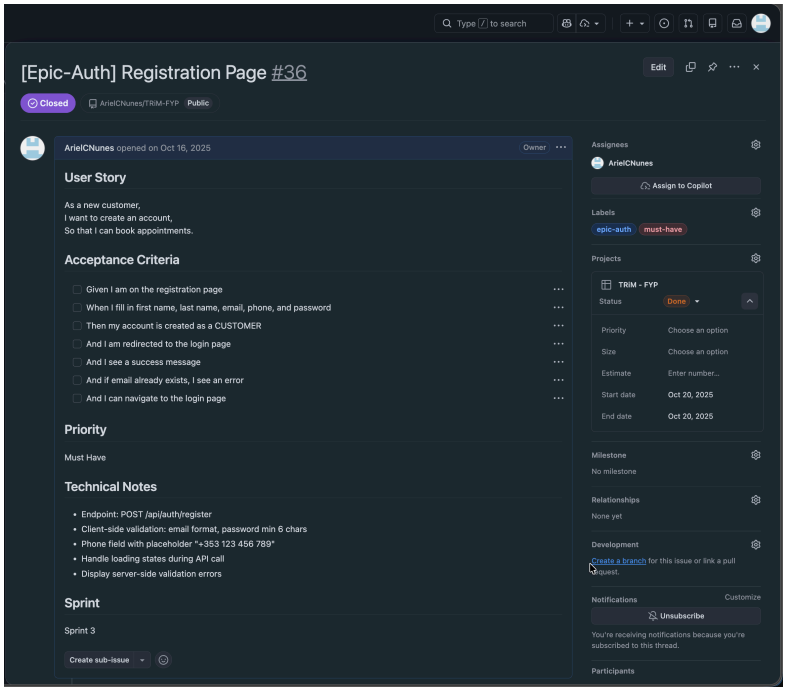

Figure 2.2: Example user story with acceptance criteria, priority, and technical notes.

Priorities were assigned based on a combination of client input, technical dependencies, and time constraints. Must Have items represented core functionality without which the system would not work, while Should Have and Could Have items were developed after the essential features were in place. Each user story mapped directly to a card on the Kanban board, providing traceability for each requirement.

2.2 Generative AI Usage

Code Generation in VSCode

GitHub Copilot was used as an inline code auto-complete in Visual Studio Code. Its primary value was in speeding up the production of boilerplate code that makes up a significant portion of any full-stack Spring Boot and React application.

On the backend, Copilot was effective at suggesting Java entity classes with JPA annotations, DTOs with getter and setter methods, repository interfaces, and repetitive controller patterns such as dependency injection, service delegation, and `ResponseEntity` wrapping. On the frontend, it assisted with TypeScript

interfaces that mirrored backend DTOs, Axios-based API service methods, and form validation logic.

This was most useful during the early stages of development when the foundational structure of entities, repositories, controllers, and base components was being established. Copilot's suggestions helped speed up this process, allowing more focus to shift toward the business logic that differentiated the system, such as the availability algorithm, deposit payment flow, and customer blacklisting. All suggestions were reviewed and adjusted before being accepted.

Testing Automation with Claude Code

Claude Code CLI was used to generate the integration test suite for the backend API. The test files were generated one at a time, with each file targeting a specific controller or operation. This allowed each test class to be reviewed individually to ensure it covered the correct functionality and edge cases.

The resulting suite consists of 15 integration test classes covering areas such as authentication, booking creation and cancellation, barber availability, payment webhooks, and dashboard endpoints. Each test class runs against an in-memory H2 database, sets up its own test data, and makes real HTTP requests through Spring's `MockMvc`. Claude Code's ability to generate the tests quickly allowed for both a much broader test suite and also quick iterations to either refine the tests or fix bugs that were found during testing.

Code Review

Claude Code was also used to review existing code for security vulnerabilities and performance issues. This involved pointing the tool at specific files or modules and having it analyse the implementation for common weaknesses such as unvalidated input, improper access control, inefficient database queries, unnecessary object creation, or even typos. Findings were not applied automatically; instead, each suggestion was worked through iteratively, discussing the reasoning behind the change before deciding whether to apply it. This back-and-forth process helped ensure the final code was both clean and well-understood.

2.3 Research Approach

Literature Search

The literature review was done across IEEE Xplore, ACM Digital Library, and ATU's library database, covering research published between 2008 and 2025. A

combination of broad and refined Boolean search queries was used, such as `multitenant database architecture`, `multi-tenant SaaS database design`, and `multi-tenan* AND (architecture OR design OR patterns)`.

Broad queries returned between 1,100 and 2,500 results, while more refined queries narrowed this to approximately 900. Duplicates across databases were removed, and the remaining papers were screened by reviewing the title, abstract, introduction, and conclusion. Papers that did not directly address database design patterns were discarded at this stage. This process resulted in a selection of 13 peer-reviewed papers.

Papers were included if they discussed database design patterns within multi-tenant architecture and addressed areas such as security, scalability, performance, or cost. Papers that were not peer-reviewed, not available in English, or that tackled multi-tenancy without focusing on database design were excluded.

Choosing the Technology Stack

The backend was built with Java and Spring Boot, chosen for its wide adoption in SaaS development and its robust security, data access, and REST APIs. Spring Security and JWT handled authentication, Spring Data JPA managed database interaction, and Spring Validation was used for input handling. PostgreSQL was selected as the database due to its support for Row-Level Security, which was central to the multi-tenant data isolation strategy explored in the literature review and applied in this project.

The frontend was built with React, TypeScript, and Vite. React was chosen for its component-based architecture, which suited the application's multiple user-facing views across customer, barber, and admin roles. TypeScript was used to catch type errors early and maintain consistency with the backend's strongly typed DTOs. Supporting libraries such as Redux Toolkit, React Router, Axios, and Tailwind CSS were selected based on compatibility with the React ecosystem. Stripe's React SDK was used for the payment integration.

Docker Compose was used to manage three containers: a PostgreSQL 16 database, the Spring Boot API, and the React frontend. This setup ensured a consistent environment between local development and the production VPS, and simplified deployment by defining the entire stack in a single configuration file.

2.4 Evaluation Methodology

Benchmarking With Gatling Load Testing

Gatling was used to load test the API under simulated real-world conditions. The simulation defines six scenarios that mirror the application's core user flows: public browsing, customer registration, the full booking lifecycle, admin management, barber operations, and an availability stress test. Each scenario combines read and write operations to reflect realistic usage rather than testing endpoints in isolation.

The load is distributed across the scenarios to approximate expected traffic patterns, with public browsing and the booking flow receiving the largest share. Users are ramped up over 60 seconds and then held at a constant level for 90 seconds. The total number of concurrent users is configurable, defaulting to 100. Assertions are set on the 95th and 99th percentile response times and the overall success rate. The simulation was run twice under the same conditions: once with Row-Level Security enabled and once without, in order to measure the performance overhead introduced by RLS. The simulation relies on a pre-seeded database, described in the following subsection. Figure 2.3 shows the simulation being executed from IntelliJ.

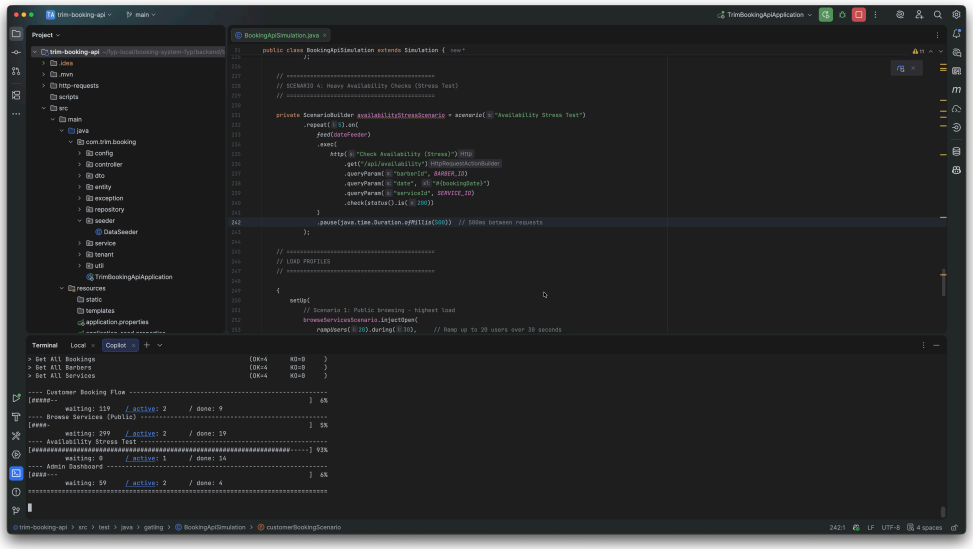

Figure 2.3: Benchmarking API in IntelliJ IDE.

Test Data Generation With DataSeeder

To run the Gatling simulation against a realistic dataset, a custom `DataSeeder` component was built using Spring Boot and the DataFaker library. The seeder is activated via a Spring profile and populates the database with configurable volumes of businesses, customers, barbers, services, and bookings using JDBC batch inserts for performance. Default values generate 5 businesses, 50,000 customers and 100,000 bookings per business, along with barbers, service categories, and realistic scheduling data such as availability windows, breaks, and booking time slots.

This seeded data served two purposes: it provided the volume needed for meaningful load testing with Gatling, and it ensured that the simulation's feeders could reference valid entities such as barber IDs, service IDs, and customer credentials.

Performance Metrics

The following metrics were used to evaluate API performance across both test runs (with and without RLS): response time at the 95th and 99th percentiles, overall success rate (percentage of requests returning expected status codes), and throughput measured in requests per second. These metrics are reported automatically by Gatling at the end of each simulation run, broken down per request, as shown in Figure 2.4. The results are explored in detail in a later chapter and were compared between the two configurations to quantify the overhead introduced by Row-Level Security.

Requests	Executions					Response Time (ms)							
	Total	OK	KO	% KO	Cnt/s	Min	50th pct	75th pct	95th pct	99th pct	Max	Mean	Std Dev
All Requests	52368	52368	0	0.00	251.77	3	14624	18023	22056	23603	25353	13777	5796
Get Active Barbers	4352	4352	0	0.00	20.92	17	14592	17761	20775	23229	24660	13754	5466
Admin Login	774	774	0	0.00	3.72	133	14160	18046	22127	23513	24556	13165	6152
Check Availability (Stress)	5600	5600	0	0.00	26.92	24	14450	17939	21965	23444	24688	13448	5939
Barber Login	1089	1089	0	0.00	5.24	112	14905	18684	22588	23732	25029	14224	5943
Customer Login	2012	2012	0	0.00	9.67	164	14238	18164	22174	23792	25353	13344	6166
Register Customer	1567	1567	0	0.00	7.53	109	14772	18040	22105	23707	24775	14220	5447
Get Active Services	4352	4352	0	0.00	20.92	211	16341	18976	22273	23603	24968	15158	5267
Get Services	2012	2012	0	0.00	9.67	10	15524	18758	22261	23602	24650	14471	5689
Get Dashboard Stats	774	774	0	0.00	3.72	762	15800	18889	22409	23806	24815	14638	5657
Get Barber Schedule	1089	1089	0	0.00	5.24	220	15919	18892	22453	23611	23953	14948	5362
Login New Customer	1567	1567	0	0.00	7.53	621	15911	18383	22046	23669	24464	15145	4945
Get Categories With Services	4352	4352	0	0.00	20.92	242	15053	18663	22374	23705	24608	14807	5205
Get Barbers	2012	2012	0	0.00	9.67	19	14039	17562	20797	23217	24413	13345	5348
Get Barber Breaks	1089	1089	0	0.00	5.24	56	14144	17877	21686	23460	23989	13636	5520
New Customer - Get Services	1567	1567	0	0.00	7.53	785	15098	18608	22438	23438	23945	14650	5340
Get All Bookings	774	774	0	0.00	3.72	19	14058	17651	20819	23092	24544	13436	5329
Check Availability	6364	6364	0	0.00	30.6	350	14303	17307	21641	23478	25322	13205	5705
Get Barber Availability	1863	1863	0	0.00	8.96	5	14202	17854	22411	23682	24661	12784	6434
Get All Barbers	774	774	0	0.00	3.72	1313	14192	16541	20733	23061	23793	12880	5541
Create Booking	2012	2012	0	0.00	9.67	2836	14352	17334	19890	22199	23935	12590	5869
Get All Services	774	774	0	0.00	3.72	2674	14376	17426	19741	22000	23898	12767	5726
Barber Check Availability	1089	1089	0	0.00	5.24	2812	14018	17434	20441	23027	23767	11915	6286
Get My Bookings	2012	2012	0	0.00	9.67	3024	14843	18956	23118	24041	24869	13573	6564
Get Categories	774	774	0	0.00	3.72	2787	15180	18887	22910	23879	25323	13689	6508
Cancel Booking	1724	1724	0	0.00	8.29	3	14288	18088	22910	23751	24666	12610	6944

Figure 2.4: Example Gatling report showing per-request execution counts and response time percentiles.

Chapter 3

Technology Review

3.1 Multi-Tenant Database Architectures

A multi-tenant system provides one shared instance of an application and its infrastructure to multiple users, referred to as tenants, while maintaining the logical separation of their data. In a SaaS context, tenants are typically organisations or businesses that each require their own private view of the data. How that separation is achieved at the database level is one of the most important design decisions in a SaaS application, as it determines the system's performance, security, operational cost, and ability to scale [1].

Different Types of Multi-Tenant Database Designs

The multi-tenant database design can be broken down into three primary isolation models, as defined by Karataş et al. [2] and illustrated in Figure 3.1.

Separate Application, Separate Database gives each tenant a fully independent application instance and database. Data isolation is guaranteed: there is no shared infrastructure at the database layer, so there is no risk of one tenant accessing another's data. The trade-off is cost and operational overhead. Every new tenant requires provisioning an additional stack, and hardware resources cannot be shared across tenants, making this model expensive to operate at scale [2].

Shared Application, Separate Database retains a single application instance but still provides each tenant with its own database. This reduces application-layer overhead while preserving data isolation. It is still expensive in terms of storage and database management, since the number of databases grows linearly with the number of tenants.

Shared Application, Shared Database is the most resource-efficient model. All tenants share the same application and the same database, with data partitioned logically rather than physically. This model has two variants. In *Shared*

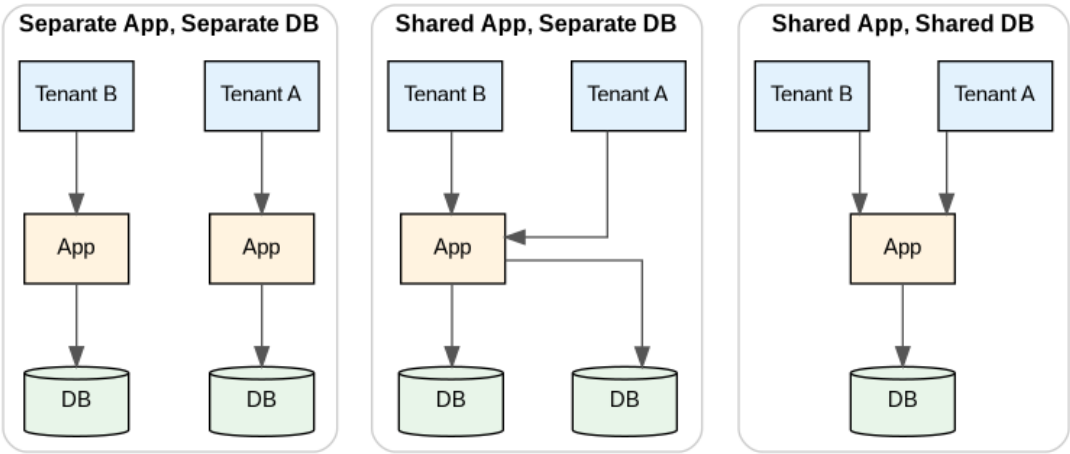

Figure 3.1: Multi-tenant database isolation patterns

Database, Separate Schema, each tenant receives its own set of tables within the shared database. In *Shared Database, Shared Schema*, all tenants share the same tables, and their rows are distinguished by a `tenant_id` column. The shared schema variant offers the highest resource utilisation and the lowest per-tenant cost, but it places a burden on the data isolation mechanisms, since a missing or incorrect filter predicate could expose one tenant's data to another [1].

Trade-offs: Isolation vs. Efficiency vs. Cost

The selection of an isolation model is not a decision that can optimise all dimensions simultaneously. Wang et al. [1] established the foundational tension: higher isolation means better security but limits the number of tenants a system can support on fixed hardware. Zhou et al. [3] quantify this directly on IBM DB2, demonstrating that a shared table model (S3) supports approximately 400 concurrent tenants on the same hardware that a separate database model (S1) can sustain only around 8. The capacity advantage of the shared approach is significant, but it depends on effective isolation being built into the system.

Cost is yet another concern. Wang et al. [1] categorise multi-tenant system costs across three dimensions: infrastructure (hardware, software, and utilisation), management (lifecycle, monitoring, and backup), and development (tenant-specific customisation). Shared architectures reduce infrastructure and management costs substantially. Aulbach et al. [4] and Yaish and Goyal [5] both demonstrate that shared schemas lower Total Cost of Ownership by eliminating per-tenant schema provisioning and by enabling self-service configuration. However, neither work

quantifies how the cost of securing a shared schema compares to the cost of stronger physical isolation.

Performance also needs to be taken into account. Wang et al. [1] measured that adding Label-Based Access Control (LBAC) - a database-level security mechanism - reduced throughput by 15% compared to less secure approaches. Ni et al. [6] propose the ADAPT methodology, which improves query performance by 2–3× over static shared-schema approaches by dynamically clustering attributes based on workload, but their evaluation explicitly assumes application-level filtering and tests no security mechanisms. The pattern is consistent across the literature: studies that optimise performance assume the security problem is solved elsewhere, and studies that address security do not measure the performance impact of their mechanisms [2, 7].

Data Isolation Techniques

Within the shared database, shared schema model, the mechanisms used to enforce tenant data isolation fall into two categories: filter-based and permission-based.

Filter-based isolation is implemented at the application layer by appending a tenant identifier predicate to every query, typically expressed as `WHERE tenant_id = ?`. Wang et al. [1] identify this as the most common approach in practice due to its simplicity. Its critical weakness is that isolation is only as reliable as the application code: a missing predicate, a buggy query, or a SQL injection vulnerability can expose data across tenant boundaries. Gupta, Kumar, and Agnihotri [8] extend this approach with session-based identifiers that rotate periodically, reducing injection risk, but their mechanism remains dependent on the application being free of errors.

Permission-based isolation enforces tenant boundaries at the database engine level, independently of the application. PostgreSQL Row-Level Security is a prominent example: policies are attached directly to tables, and the database engine automatically appends the isolation predicate to every qualifying query before execution, regardless of what the application sends. Factor et al. [9] formalise the principles underpinning this approach in their SLIM (Secure Logical Isolation for Multi-tenancy) model, based on least privilege, tenant containment, escalation avoidance, and minimising attack surface. The advantage of database-level isolation is that it acts as a safety net: even if the application layer has a bug, the database will not return rows belonging to another tenant. The documented trade-off is a performance overhead from the additional policy evaluation on every query.

3.2 Related Works

This section examines specific studies that have proposed techniques for managing multi-tenant data and prior systematic reviews that establish the broader research context.

Proposed Multi-Tenant SaaS Database Design Patterns

Several studies have proposed specific techniques for managing multi-tenant data that are directly relevant to the design decisions made in this project.

Ni et al. [6] introduce **ADAPT** (Adaptive Database Schema Design for Multi-Tenant Data Management), a methodology that dynamically reorganises table schemas by clustering attributes based on observed query workloads. ADAPT achieves a 2–3× throughput improvement over static shared-schema designs and demonstrates reasonable scalability up to 4,000 tenants, at which point the alternative individual-table approach degrades severely. Its limitation for this project's context is that it assumes application-level filtering and does not evaluate how it would work with database-level security policies.

Mangwani and Tokekar [10] evaluate microservice architectures as a scalability mechanism for multi-tenant SaaS systems. Their findings show that at low concurrency (around 50 users), monolithic and microservice architectures perform similarly, but at high load (300+ users), microservices offer lower response times, higher throughput, and approximately 11% better CPU utilisation. This supports the case for modular application design but treats the database as a black box, with no evaluation of how schema design choices interact with service decomposition.

Sharma and Kaur [11] demonstrate MongoDB sharding as an alternative to relational multi-tenant designs, enabling horizontal scaling through data partitioning across nodes with automatic load balancing. While this illustrates the growing relevance of NoSQL approaches, the study does not compare its performance or isolation guarantees against optimised relational alternatives, leaving the trade-off between schema flexibility and data isolation unknown.

Literature Reviews

Two prior systematic reviews are most directly relevant to this work and establish the context into which this project's design decisions fit.

Karataş et al. [2] conducted a broad systematic mapping study covering 407 papers from 2000 to 2017. Their key quantitative finding is that 65% of papers preferred a shared database strategy, and 58% were ideological proposals without implementation. The study is descriptive rather than evaluative: it reports which strategies are preferred but does not analyse why or evaluate combined approaches.

Its 2017 cutoff also means it predates significant developments in containerised deployment and modern database security tooling.

Battula [7] provides a more recent (2024) systematic review focused specifically on security mechanisms in multi-tenant cloud databases, analysing 25 papers from 2012 to 2024. It comprehensively covers encryption, access control, and threat detection, but treats performance and scalability as secondary concerns. Like Karatąs et al., it evaluates mechanisms in isolation and identifies no integrated approaches combining database-level security with performance optimisation techniques.

The gap identified by both reviews, and confirmed by the broader set of papers analysed, is the absence of empirical evaluation of combined approaches. No study evaluates how a database-level security mechanism such as RLS performs when applied alongside a shared schema optimised for multi-tenant query patterns, nor how much of the performance benefit claimed by approaches like ADAPT survives the addition of policy-based isolation. Zhu et al. [12] highlight an additional dimension of this problem: optimising database performance for one tenant can degrade it for others, a conflict that existing research leaves unresolved.

This project addresses a subset of this integration gap directly. TRiM adopts a Shared Application, Shared Database, Shared Schema design, the pattern associated with the highest resource efficiency, and enforces tenant isolation using PostgreSQL Row-Level Security rather than application-layer filtering alone. The load testing results presented in Chapter 5 provide empirical evidence on the performance overhead of this combination under real data volumes, contributing practical findings to the field.

Chapter 4

System Design

This chapter describes the architecture and design decisions behind *TRiM*. It covers the overall system structure, multi-tenant data isolation strategy, core business logic, frontend design, and deployment set up.

4.1 System Overview

TRiM is built as a multi-tenant Software-as-a-Service (SaaS) platform following a three-tier architecture: a React single-page application on the frontend, a Spring Boot REST API on the backend, and a PostgreSQL database for persistence. Each tenant (i.e. each business using the platform) is identified by a unique subdomain such as `v7.trimbooking.ie`, and all three tiers are tenant-aware, which means every request is scoped to a single business from the moment it enters the system.

The frontend is a TypeScript React app bundled with Vite and served by Nginx. It communicates with the backend through a RESTful JSON API. The backend is a stateless Spring Boot application that handles authentication, business logic, and data access. Authentication is token-based using JSON Web Tokens (JWT), with no server-side session state. As discussed earlier, PostgreSQL was selected specifically for its native support for RLS, which enforces tenant isolation at the database level and ensures that even a bug in application code cannot expose businesses' data.

Integrations include Stripe Connect for payment processing, where each business connects their own Stripe account to receive deposits directly, and Twilio and SMTP for sending booking confirmation notifications via SMS and email.

The system is containerised using Docker Compose, with three services: the PostgreSQL database, the Spring Boot API, and the Nginx-served frontend. In production, all three containers run on a Virtual Private Server (VPS), an Infrastructure-as-a-Service (IaaS) deployment, with Nginx acting as a reverse proxy.

Architecture Diagram

Figure 4.1 illustrates the high-level architecture of the system, showing the request flow from the browser through Nginx and the Spring Boot filter chain to PostgreSQL, along with the external service integrations.

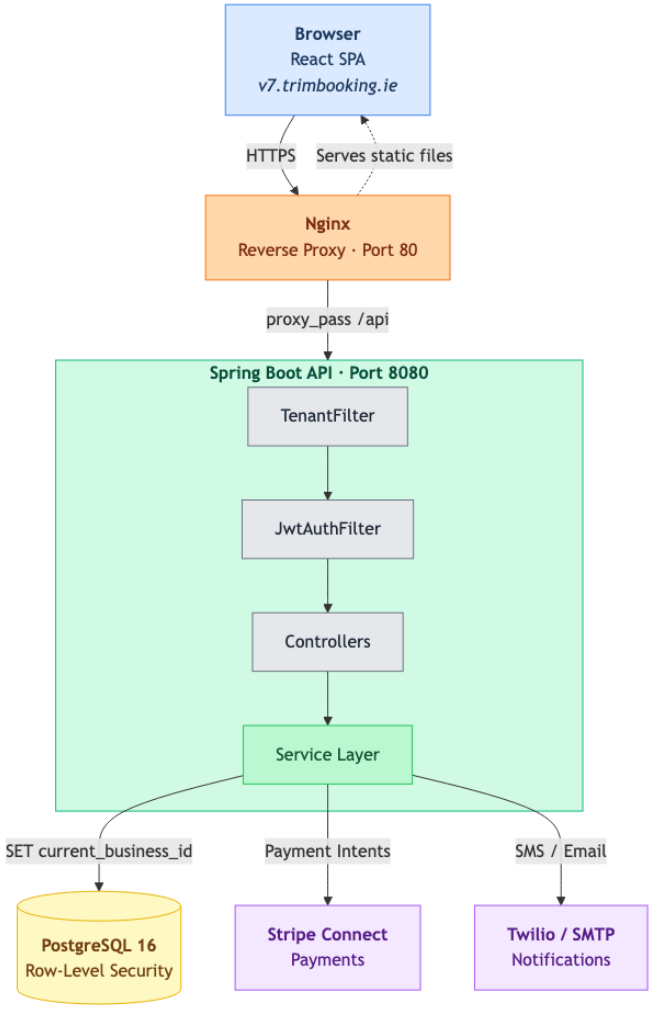

Figure 4.1: High-level system architecture diagram.

RESTful API Design

The backend exposes a RESTful JSON API under the `/api` prefix. All endpoints follow standard REST conventions: resources are identified by URL paths, operations map to HTTP methods (GET, POST, DELETE, etc.), and all request and response bodies use JSON. The API is entirely stateless and there are no

server-side sessions. Every authenticated request requires a JWT passed in the `Authorization` header.

Endpoints are split into two categories. Public endpoints, such as login, registration, active service listings, availability lookups, and the Stripe webhook, can be accessed without authentication. All remaining endpoints require a valid JWT, and Spring Security rejects any request that does not carry one. Because the system is multi-tenant, every request also carries the business context implicitly through the subdomain or `X-Business-Slug` header. No endpoint requires a `businessId` parameter. The `TenantFilter` resolves it automatically before the request reaches any controller.

Listing 4.1 shows the Axios request interceptor on the frontend, which attaches both the JWT and the business slug header to every outgoing API call.

Listing 4.1: Axios request interceptor attaching JWT and tenant context to every API call.

```
// Request interceptor - Add JWT token and business slug to every
    request
api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem("token");
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }

    // Send business slug header only when present
    const businessSlug = getBusinessSlug();
    if (businessSlug) {
      config.headers["X-Business-Slug"] = businessSlug;
    }

    return config;
  },
  (error) => {
    return Promise.reject(error);
  },
);
```

Table 4.1 summarises the main API endpoint groups exposed by the Java backend.

Resource	Prefix	Description
Authentication	`/api/auth`	Login, register, password reset, cross-subdomain token exchange
Bookings	`/api/bookings`	Create, reschedule, cancel, mark complete or no-show
Barbers	`/api/barbers`	Create, update, activate and deactivate barber profiles
Barber Breaks	`/api/barber-breaks`	Manage scheduled break times per barber
Availability	`/api/availability`	Query available time slots for a given barber, date, and service
Barber Availability	`/api/barber-availability`	Set and update weekly working hours per barber
Services	`/api/services`	Create, update, and deactivate services offered
Categories	`/api/categories`	Manage service categories with nested service listings
Payments	`/api/payments`	Create Stripe payment intents and handle webhooks
Stripe Connect	`/api/stripe-connect`	Business onboarding, account status, dashboard access
Customers	`/api/admin/customers`	List customers, blacklist and unblacklist
Dashboard	`/api/dashboard`	Admin statistics and revenue metrics

Table 4.1: Summary of API endpoint groups.

4.2 Multi-Tenant Database Design

This section describes the multi-tenant data isolation strategy used in *TRiM*, which is a critical aspect of the system's design. The main goal is to ensure that each tenant's data is completely isolated and secure, while still allowing for efficient querying and scalability.

Entity-Relationship Model

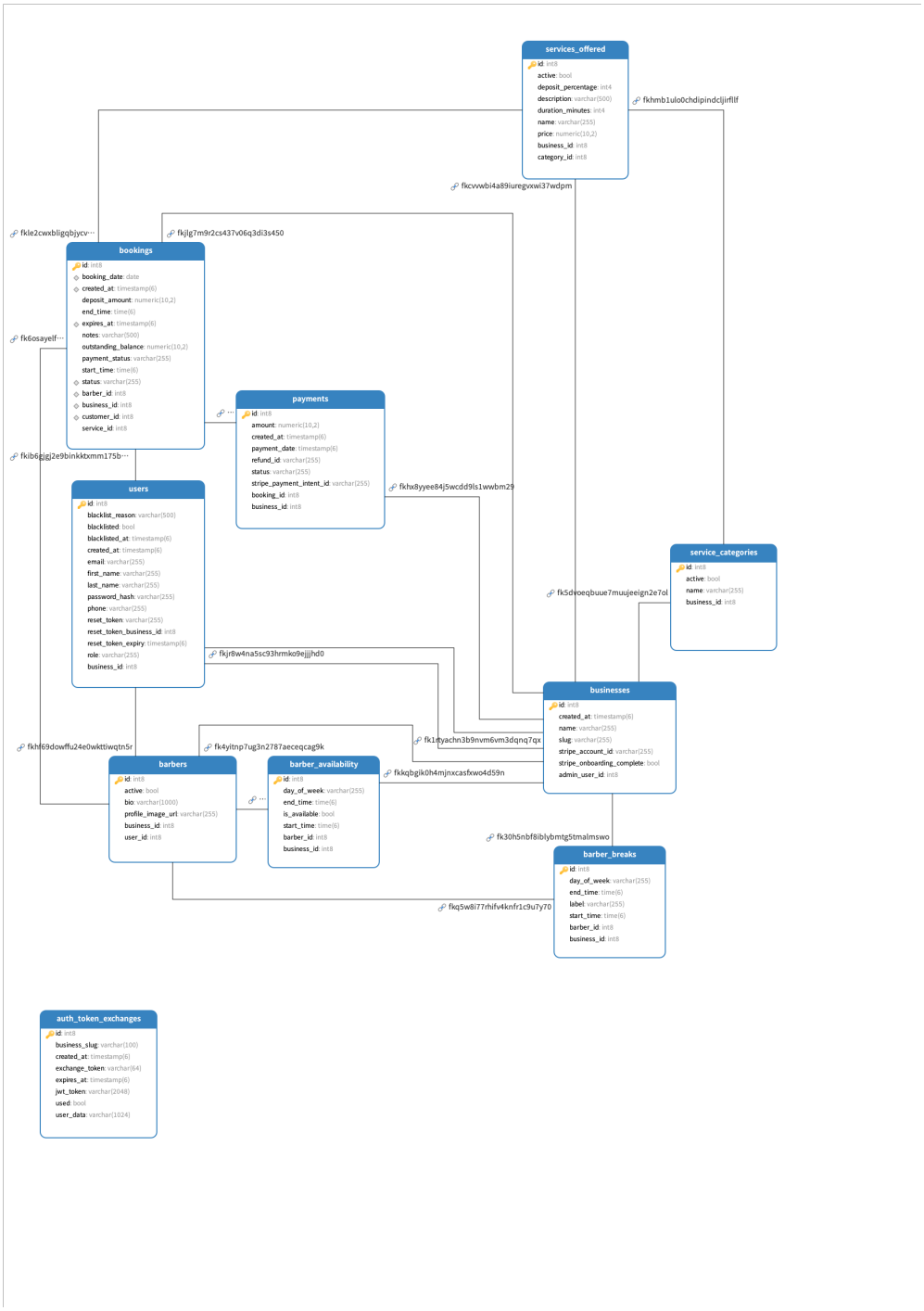

Figure 4.2: Entity-Relationship diagram of the PostgreSQL database schema.

Figure 4.2 shows the full database schema. The central entity is `businesses`, which represents a tenant. Nearly every other table carries a `business_id` foreign key that ties its rows to a specific business, forming the foundation of the multi-tenant isolation model.

The core domain relationships are as follows. A `booking` links a customer (`users`), a `barber`, and a `service_offered`, along with the date, time slot, and status. Each booking may have an associated `payment` record that tracks the Stripe payment intent and deposit amount. Barbers have two supporting tables: `barber_availability` defines their weekly working hours, and `barber_breaks` defines scheduled breaks within those hours. Services are grouped into `service_categories`, with each service belonging to one category.

The `users` table stores all user types (customers, barbers, and admins) distinguished by a `role` column. Barbers are linked to their user account via a `user_id` foreign key, while the business's admin is referenced by `admin_user_id` on the `businesses` table.

The only table without a `business_id` is `auth_token_exchanges`, which stores short-lived tokens used for cross-subdomain authentication. These tokens operate across tenants by design, as they facilitate the redirect from the main registration page to a tenant-specific subdomain.

Migrating From Single to Multi-Tenant

The system was initially developed as a single-tenant application, with all entities (users, barbers, bookings, services) existing in a single flat schema with no concept of business ownership. Once the core functionality was done, the codebase was refactored to support multiple tenants. This involved some key changes:

- A new `businesses` table was introduced to represent each tenant, with a unique `slug` used for subdomain-based identification.
- A `business_id` foreign key column was added to every tenant-scoped table, linking each row to its owning business.
- JPA entity classes were updated to include a `@ManyToOne` relationship to `Business`, and repository queries were updated to filter by `business_id`.
- The JWT token payload was extended to include a `businessId` claim, so that each token is bound to a specific tenant.
- A request filter chain was introduced (`TenantFilter` and `JwtAuthenticationFilter`) to resolve and validate the tenant context on every request, as described later in this chapter.

- PostgreSQL Row-Level Security policies were applied to enforce isolation at the database level, independent of application code.

This incremental approach meant that the single-tenant version served as a working prototype, and the multi-tenant refactor could be layered on top without redesigning the core logic.

4.3 Multi-Tenant Implementation

A common approach to multi-tenant isolation is to filter data purely in application code, for example by appending `WHERE business_id = ?` to every query. While straightforward, this approach is fragile: a single forgotten filter or a new repository method without a tenant check is enough to leak data across businesses.

As identified in the literature review (Chapter 3), PostgreSQL Row-Level Security (RLS) offers a stronger alternative. RLS policies are defined at the database level and automatically filter every `SELECT`, `INSERT`, `UPDATE`, and `DELETE` operation based on a session variable. The application cannot bypass these policies, regardless of how the query is constructed.

TRiM uses both approaches together as a defence-in-depth strategy, summarised in Table 4.2. Any single layer is sufficient to prevent cross-tenant data access; together they ensure that even if one layer fails, the others still enforce isolation. The remainder of this section describes how each layer is implemented.

Layer	**Mechanism**	**What it prevents**
Application	Repository methods filter by `business_id`	Queries are scoped to the current tenant by design
Authentication	JWT `businessId` must match `TenantContext`	Tokens issued for one business cannot be used on another
Database	PostgreSQL RLS policies	Even if both application layers fail, the database will not return rows outside the current tenant

Table 4.2: Three independent layers of tenant data isolation.

How Is a Request Scoped to a Tenant?

Every incoming request must be associated with a specific business before any controller logic runs. This is handled by two components working together: `TenantContext`, a thread-local store, and `TenantFilter`, a request filter.

`TenantContext` is a utility class that uses Java's `ThreadLocal` to hold the current business ID and slug for the duration of a single HTTP request. It is written to by the `TenantFilter` at the start of the request, read by the `RlsDataSource` when a database connection is checked out, and cleared in a `finally` block to prevent memory leaks.

`TenantFilter` is a request filter that runs before the JWT authentication filter in the security filter chain. It resolves which business the request is for using three strategies, in this order:

1. **Subdomain** – the first part of the hostname is extracted (e.g. `v7.trimbooking.ie` yields `v7`). Common subdomains such as `www`, `api`, and `localhost` are ignored.

2. **Header** – the `X-Business-Slug` header, sent by the frontend's Axios interceptor on every request.

3. **Query parameter** – a `?business=slug` fallback for development and testing.

Once a slug is resolved, the filter looks up the `Business` entity by slug. If found, it calls `TenantContext.setCurrentBusiness()` and the request proceeds. If no slug is provided or the business does not exist, the request is rejected with a 400 or 404 response before it reaches any controller.

Certain endpoints are exempted from tenant resolution because they operate outside the scope of a single business such as admin registration (which creates a new business), the cross-subdomain token exchange endpoint, and the Stripe payment webhook.

Listing 4.2 shows the core of the filter's resolution logic.

Listing 4.2: TenantFilter extracting business slug from multiple sources.

```java
private String extractBusinessSlug(HttpServletRequest request) {
    // Strategy 1: Extract from subdomain
    String slug = extractFromSubdomain(request);
    if (slug != null && !slug.isEmpty()) return slug;

    // Strategy 2: X-Business-Slug header
    slug = request.getHeader("X-Business-Slug");
    if (slug != null && !slug.isEmpty()) return slug;

    // Strategy 3: Query parameter
    slug = request.getParameter("business");
    if (slug != null && !slug.isEmpty()) return slug;

    return null;
}
```

Cross-Subdomain Token Exchange

When a new admin registers on the main site, the system creates their business and issues a JWT token. The user then needs to be redirected to their tenant-specific subdomain (e.g. `v7.trimbooking.ie`). However, the JWT cannot simply be passed along: cookies do not transfer across subdomains, and placing a long-lived token in a URL query parameter would be insecure as it could be leaked through browser history or server logs.

To solve this, *TRiM* uses a short-lived, single-use exchange token. The flow works as follows:

1. After registration, the backend generates a cryptographically secure random token (32 bytes, Base64 URL-encoded) and stores it in the `auth_token_exchanges` table alongside the actual JWT, the user data, and the target business slug. The exchange token expires after 60 seconds.

2. The user is redirected to the target subdomain with the exchange token in the URL (e.g. `v7.trimbooking.ie/auth?token=abc123`).

3. The frontend on the target subdomain sends the exchange token to the `/api/auth/exchange-token` endpoint, which looks up the token, validates that it has not expired or been used, marks it as used, and returns the actual JWT and user data.

4. The frontend stores the JWT in `localStorage` and the user is authenticated on the new subdomain.

This approach ensures that the actual JWT is never exposed in a URL. The exchange token is single-use, short-lived, and cryptographically random, making it safe to pass as a query parameter. A scheduled task cleans up expired and used tokens every five minutes.

4.4 Business Operations

This section describes the core business logic of the platform: how a customer books an appointment, how deposits are collected through Stripe, and how the system prevents two customers from booking the same time slot simultaneously.

Booking & Payment Flow

The booking and payment process in *TRiM* follows a three-phase flow: the customer creates a booking, pays a deposit through Stripe, and the system confirms the booking once payment succeeds. Each phase is described below.

Phase 1: Booking Creation

When a customer submits a booking request, the backend runs through a series of validation and conflict-detection steps inside the database transaction:

1. **Entity validation**: the customer, barber, and service are loaded from the database and verified to exist and belong to the same business.
2. **Blacklist check**: if the customer has been blacklisted by the business, the request is rejected.
3. **Time validation**: the requested date and time must be in the future.
4. **Conflict detection**: the system acquires a pessimistic lock on all existing bookings for that barber and date, then checks for time overlaps. This step is described in the next subsection.
5. **Booking saved**: if no conflicts are found, the booking is persisted with a status of `PENDING` and an expiry time set to ten minutes from now.

The ten-minute expiry is intentional. A pending booking holds a time slot, preventing other customers from booking it. If, for some reason, the payment is not completed within ten minutes, the booking expires and the slot is released. Expired bookings are excluded from conflict detection and are periodically cleaned up by a scheduled task that marks them as `CANCELLED`.

Phase 2: Deposit Payment via Stripe

After the booking is created, the frontend calls `/api/payments/create-intent` to initiate the deposit payment. The backend calculates the deposit as a percentage of the service price (configured per service and set by the ADMIN user), verifies that the business has completed Stripe Connect onboarding, and creates a Stripe `PaymentIntent` on the business's connected account. The key point is that *TRiM* uses Stripe Connect in its Standard model: each business connects their own Stripe account during onboarding, and deposits are paid directly into that account. The platform does not hold customer funds.

The `PaymentIntent` includes the `booking_id` and `business_id` in its metadata, which are used later for webhook verification. The backend returns the Stripe `clientSecret` to the frontend, which renders the Stripe payment form using Stripe Elements. The customer completes the payment (card, Apple Pay, or Google Pay) entirely on the client side.

Phase 3: Webhook Confirmation

Once Stripe processes the payment, it sends a `payment_intent.succeeded` event to the `/api/payments/webhook` endpoint. The webhook handler performs several verification steps before confirming the booking:

1. **Signature verification**: the `Stripe-Signature` header is validated against the webhook secret to ensure the request genuinely came from Stripe.
2. **Cross-tenant verification**: the `business_id` from the payment metadata is compared against the booking's actual business, and the Stripe connected account ID on the event is compared against the business's stored account ID. A mismatch in either check rejects the webhook.
3. **Booking confirmation**: the payment record is marked as `SUCCEEDED`, the booking status transitions from `PENDING` to `CONFIRMED`, the deposit amount is recorded, and the ten-minute expiry is cleared.
4. **Notifications**: a confirmation email and SMS are sent to the customer with the booking details.

Listing 4.3 shows the `handlePaymentSuccess` method that executes steps 2–4. Because the webhook originates from Stripe rather than from a tenant-scoped browser request, the method manually sets the RLS context using the business ID from the payment metadata before performing any database queries.

Listing 4.3: Webhook handler confirming a booking after successful deposit payment.

```java
@Transactional
public void handlePaymentSuccess(String paymentIntentId,
                                 Long businessIdFromMetadata) {
    // Set RLS context manually (webhook has no tenant filter)
    entityManager.createNativeQuery(
        "SELECT set_config('app.current_business_id', ?1, true)")
        .setParameter(1, String.valueOf(businessIdFromMetadata))
        .getSingleResult();

    Payment payment = paymentRepository
        .findByStripePaymentIntentId(paymentIntentId)
        .orElseThrow(() -> new RuntimeException("Payment not found
            "));

    Booking booking = payment.getBooking();

    // Verify business_id from metadata matches booking's business
```

```
    if (!businessIdFromMetadata.equals(booking.getBusiness().getId
        ())) {
        throw new RuntimeException(
            "Payment business verification failed");
    }

    // Update payment and booking status
    payment.setStatus(Payment.PaymentStatus.SUCCEEDED);
    payment.setPaymentDate(LocalDateTime.now());
    paymentRepository.save(payment);

    booking.setPaymentStatus(Booking.PaymentStatus.DEPOSIT_PAID);
    booking.setStatus(Booking.BookingStatus.CONFIRMED);
    booking.setExpiresAt(null);
    bookingRepository.save(booking);

    // Send confirmation notifications
    emailService.sendBookingConfirmation(booking);
    smsService.sendBookingConfirmation(booking);
}
```

Race Condition Prevention When Booking

A race condition occurs when two or more concurrent requests read the same state and both act on it before either write is committed. In a booking system, this means two customers could simultaneously check availability for the same barber at the same time slot, both see it as free, and both successfully create a booking, resulting in a double-booking. This is known as a time-of-check-to-time-of-use (TOCTOU) problem.

TRiM prevents this using pessimistic locking at the database level. When a booking is being created, the conflict detection query acquires an exclusive write lock on all existing booking rows for that barber and date. This is done using JPA's `@Lock` annotation, which translates to a PostgreSQL `SELECT ... FOR UPDATE` statement. While one transaction holds this lock, any other transaction attempting to book the same barber on the same date will block until the first transaction either commits or rolls back. This serialises concurrent booking attempts and eliminates the race condition entirely.

Listing 4.4 shows the repository method that acquires the lock.

Listing 4.4: Pessimistic write lock on booking rows for conflict detection.

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT b FROM Booking b " +
       "WHERE b.business.id = :businessId " +
       "AND b.barber.id = :barberId " +
```

```
        "AND b.bookingDate = :bookingDate")
List<Booking> findByBusinessIdAndBarberIdAndBookingDateWithLock(
    @Param("businessId") Long businessId,
    @Param("barberId") Long barberId,
    @Param("bookingDate") LocalDate bookingDate);
```

Once the rows are locked and returned, the `BookingConflictDetectionService` iterates over them and checks for time overlaps. Cancelled bookings and expired pending bookings are skipped, since those slots are available for reuse. The overlap check uses a standard interval comparison: two time slots overlap if and only if the first starts before the second ends and the first ends after the second starts.

Listing 4.5 shows the conflict detection logic.

Listing 4.5: Time slot conflict detection with pessimistic locking.

```
public void validateTimeSlotAvailable(Long barberId,
        LocalDate bookingDate, LocalTime startTime, LocalTime
            endTime) {
    List<Booking> existingBookings = bookingRepository
        .findByBusinessIdAndBarberIdAndBookingDateWithLock(
            getBusinessId(), barberId, bookingDate);

    for (Booking existing : existingBookings) {
        if (existing.getStatus() == BookingStatus.CANCELLED)
            continue;
        if (existing.isExpired()) continue;

        // Overlap: start1 < end2 AND end1 > start2
        if (startTime.isBefore(existing.getEndTime())
                && endTime.isAfter(existing.getStartTime())) {
            throw new ConflictException(
                "This time slot is no longer available");
        }
    }
}
```

It is worth noting that the availability endpoint used by the frontend to display open slots does *not* acquire a lock, it performs a regular read query. This is intentional: locking during availability checks would degrade performance for a read-heavy operation that does not modify data. The lock is only acquired at the point of booking creation, where correctness is critical. If a slot becomes unavailable between the time the customer views it and the time they submit, the conflict detection will reject the request and the frontend displays an error prompting them to choose another time.

4.5 Authentication & Access Control

This section describes how *TRiM* authenticates users and controls what each user is allowed to do and see. Authentication is handled entirely through JSON Web Tokens with no server-side sessions, and access is restricted based on three user roles: customer, barber, and admin.

JWT Structure & Validation

Every authenticated request in *TRiM* carries a JSON Web Token in the HTTP `Authorization` header as a Bearer token. The token is generated at login by the `JwtUtil` component and contains four claims: the user's email (as the JWT subject), their role, their user ID, and their business ID. Tokens are signed using HMAC-SHA256 with a secret key provided through an environment variable. At startup, the application validates that the secret is configured and is at least 256 bits long, rejecting deployment if either check fails. Tokens expire after 24 hours.

The `JwtAuthenticationFilter` runs on every incoming request, after the `TenantFilter` has already established the tenant context. It extracts the token from the header, verifies the signature and expiry, and then performs a cross-tenant check: the `businessId` claim in the token must match the business ID resolved by the `TenantFilter`. If they do not match, the request is rejected with a 403 response. This prevents a token issued for one business from being used on another business's subdomain, forming the authentication layer of the three-layer tenant isolation strategy described in Table 4.2.

If the token is valid, the filter creates a Spring Security `Authentication` object containing the user's email as the principal, a granted authority of `ROLE_CUSTOMER`, `ROLE_BARBER`, or `ROLE_ADMIN` derived from the token's role claim, and the user ID stored in the authentication details. This object is placed into the `SecurityContext`, making the user's identity and role available to all downstream controllers and security checks for the duration of the request.

Listing 4.6 shows the core of the JWT validation and tenant cross-check in the authentication filter.

Listing 4.6: JWT authentication filter with multi-tenant validation.

```
if (jwtUtil.isTokenValid(token)) {
    String email = jwtUtil.extractEmail(token);
    String role = jwtUtil.extractRole(token);
    Long userId = jwtUtil.extractUserId(token);
    Long tokenBusinessId = jwtUtil.extractBusinessId(token);

    // Reject if token's business doesn't match tenant context
    Long contextBusinessId = TenantContext.getCurrentBusinessId();
    if (tokenBusinessId == null || contextBusinessId == null
```

```
            || !tokenBusinessId.equals(contextBusinessId)) {
        response.setStatus(HttpServletResponse.SC_FORBIDDEN);
        response.getWriter().write(
            "{\"error\": \"Token not valid for this business\"}");
        return;
    }

    // Set authentication in Spring SecurityContext
    SimpleGrantedAuthority authority =
        new SimpleGrantedAuthority("ROLE_" + role);
    UsernamePasswordAuthenticationToken authentication =
        new UsernamePasswordAuthenticationToken(
            email, null, Collections.singletonList(authority));
    authentication.setDetails(userId);
    SecurityContextHolder.getContext()
        .setAuthentication(authentication);
}
```

Role-Based Access Control

TRiM defines three user roles: `CUSTOMER`, `BARBER`, and `ADMIN`. A customer is any user who registers through a business's subdomain to make bookings. A barber is created by an admin and represents a service provider within the business. An admin is the business owner who registered the business on the platform. Access control is enforced at three levels.

URL-Level Security

The `SecurityConfig` class defines which endpoints are publicly accessible and which require authentication. Public endpoints include login, registration, active service and barber listings, availability lookups, and the Stripe webhook. All other endpoints require a valid JWT. This is configured through Spring Security's `authorizeHttpRequests` builder, with `anyRequest().authenticated()` as the default rule.

Method-Level Security

Individual controller methods use Spring's `@PreAuthorize` annotation to restrict access by role. For example, the endpoint to view all bookings is annotated with `@PreAuthorize("hasRole('ADMIN')")`, limiting it to business administrators. Endpoints for managing a barber's schedule, marking bookings as completed, or recording no-shows require `hasAnyRole('BARBER', 'ADMIN')`, allowing both the barber and the admin to perform these operations. Table 4.3 summarises the role requirements for the main endpoint groups.

Endpoint Group	Required Role	Description
All bookings	ADMIN	View and filter all bookings for the business
Barber schedule	BARBER, ADMIN	View bookings, mark complete, no-show, paid
Customer management	ADMIN	List customers, blacklist and unblacklist
Service management	ADMIN	Create, update, and deactivate services
Barber management	ADMIN	Create, update, and deactivate barbers
Dashboard	ADMIN	View statistics and revenue metrics
Barber availability	BARBER, ADMIN	Set and update weekly working hours
Create booking	Any authenticated	Customers book appointments

Table 4.3: Role requirements for protected API endpoint groups.

Resource-Level Ownership

Role checks alone are not sufficient for all cases. A customer with a valid token should not be able to view another customer's bookings, and a barber should not be able to modify another barber's schedule. To handle this, controllers perform programmatic ownership checks after authentication. The authenticated user's ID is extracted from the `SecurityContext` and compared against the resource being accessed. For example, when a customer requests their bookings, the controller verifies that the `customerId` in the URL matches the authenticated user's ID. If it does not, the request is rejected with a 403 response. Admins are exempt from these checks and can access any resource within their business.

4.6 Frontend Architecture

Component Structure

State Management

Responsive Design

Integration with Backend API

4.7 Deployment Architecture

Docker

Deploying Application to a VPS

Chapter 5

System Evaluation

5.1 Performance Evaluation

5.1.1 Database Gaitling Benchmarking - Load Testing

5.2 Security Evaluation

5.2.1 Tenant Isolation Verification

5.3 Critical Analysis

5.3.1 Strengths

5.3.2 Limitations

Chapter 6

Conclusion

6.1 Future Work

Briefly summarise your context and objectives. Remind the reader about the overall rationale and goals of the project. Highlight your findings from the System Evaluation chapter.

Bibliography

[1] Zhi Hu Wang, Chang Jie Guo, Bo Gao, Wei Sun, Zhen Zhang, and Wen Hao An. A Study and Performance Evaluation of the Multi-Tenant Data Tier Design Patterns for Service Oriented Computing. In *2008 IEEE International Conference on E-Business Engineering*, pages 94–101.

[2] Gözde Karata, Ferit Can, Gamze Doan, Cemile Konca, and Akhan Akbulut. Multi-tenant architectures in the cloud: A systematic mapping study. In *2017 International Artificial Intelligence and Data Processing Symposium (IDAP)*, pages 1–4.

[3] Ying Hua Zhou, Qi Rong Wang, Zhi Hu Wang, and Ning Wang. DB2MMT: A Massive Multi-tenant Database Platform for Cloud Computing. In *2011 IEEE 8th International Conference on E-Business Engineering*, pages 335–340.

[4] Stefan Aulbach, Michael Seibold, Dean Jacobs, and Alfons Kemper. Extensibility and Data Sharing in evolving multi-tenant databases. In *2011 IEEE 27th International Conference on Data Engineering*, pages 99–110.

[5] Haitham Yaish and Madhu Goyal. A Multi-tenant Database Architecture Design for Software Applications. In *2013 IEEE 16th International Conference on Computational Science and Engineering*, pages 933–940.

[6] Jiacai Ni, Guoliang Li, Lijun Wang, Jianhua Feng, Jun Zhang, and Lei Li. Adaptive Database Schema Design for Multi-Tenant Data Management. 26(9):2079–2093.

[7] Moshe Battula. A Systematic Review on a Multi-tenant Database Management System in Cloud Computing. In *2024 International Conference on Cognitive Robotics and Intelligent Systems (ICC - ROBINS)*, pages 890–897.

[8] Keshav Gupta, Sandeep Kumar, and Ojaswi Agnihotri. Data isolation in multi-tenant SaaS environment. In *2016 International Conference on Computing, Communication and Automation (ICCCA)*, pages 1290–1292.

[9] Michael Factor, David Hadas, Aner Harnama, Nadav Har'el, Elliot K. Kolodner, Anil Kurmus, Alexandra Shulman-Peleg, and Alessandro Sorniotti. Secure Logical Isolation for Multi-tenancy in cloud storage. In *2013 IEEE 29th Symposium on Mass Storage Systems and Technologies (MSST)*, pages 1–5.

[10] Poonam Mangwani and Vrinda Tokekar. Container Based Scalability and Performance Analysis of Multitenant SaaS Applications. In *2022 13th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, pages 1–6.

[11] Aditi Sharma and Parmeet Kaur. Implementing a Multitenant System using a Document-based NoSQL Database. In *2021 5th International Conference on Information Systems and Computer Networks (ISCON)*, pages 1–5.

[12] Zekai Zhu, Peiquan Jin, Guorui Huang, Xiaoliang Wang, and Yigui Yuan. MTRP: A High-Performance Cost-Efficient Buffering Scheme for Multi-Tenant Cloud Services. In *2024 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA)*, pages 1677–1682.